

Học viện công nghệ bưu chính viễn thông

Khoa đào tạo sau đại học



BÁO CÁO BÀI TẬP LỚN

Bộ môn: Các Hệ Thống Phân Tán

**CHỦ ĐỀ 3: PHÁT TRIỂN HỆ THỐNG CHAT NGANG
HÀNG P2P**

(PEER-TO-PEER CHAT SYSTEM)

Giảng viên hướng dẫn

TS. Kim Ngọc Bách

Sinh viên/Học viên

Trần Đình Dũng - B25CHHT089

Công nghệ sử dụng

Node.js, TCP Socket, Express.js,
Socket.IO

Mục Lục

DANH MỤC TỪ VIẾT TẮT.....	5
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	6
1.1. Giới thiệu đề tài	6
1.2. Mục tiêu thực hiện.....	6
1.3. Phạm vi hệ thống.....	6
1.4. Tóm tắt kết quả đạt được.....	7
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	8
2.1. Hệ thống phân tán.....	8
2.2. Mô hình Peer-to-Peer	8
2.3. TCP Socket.....	8
2.4. Peer Discovery và Heartbeat	8
2.5. Công nghệ sử dụng.....	9
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU HỆ THỐNG	10
3.1. Yêu cầu chức năng theo đề bài.....	10
3.2. Yêu cầu kỹ thuật.....	10
3.3. Chức năng nâng cao	11
3.4. Yêu cầu phi chức năng	12
CHƯƠNG 4. KIẾN TRÚC VÀ THIẾT KẾ HỆ THỐNG	13
4.1. Kiến trúc tổng thể	13
4.2. Bootstrap/Tracker Server	13
4.3. Peer Node	13
4.4. Cấu trúc thư mục và module	14
4.5. Luồng hoạt động tổng quát.....	14

CHƯƠNG 5. GIAO THỨC TRAO ĐỔI THÔNG ĐIỆP.....	16
5.1. Giao thức REST giữa Peer và Bootstrap Server	16
5.2. Giao thức TCP giữa các peer.....	16
5.3. Các loại thông điệp.....	17
5.4. Mẫu JSON thông điệp	17
5.5. Luồng gửi tin nhắn trực tiếp.....	18
CHƯƠNG 6. CÁC CƠ CHẾ XỬ LÝ PHÂN TÁN VÀ CHỊU LỖI	19
6.1. Cơ chế peer discovery	19
6.2. Quản lý trạng thái online/offline	19
6.3. Cơ chế truyền tin đáng tin cậy.....	20
6.4. Khử trùng lặp tin nhắn.....	20
6.5. Store-and-Forward khi peer offline.....	20
6.6. Mã hóa tin nhắn AES-256-CBC.....	21
6.7. Broadcast và chat nhóm	21
6.8. Mô phỏng churn	21
CHƯƠNG 7. GIAO DIỆN NGƯỜI DÙNG VÀ HƯỚNG DẪN CHẠY.....	23
7.1. Giao diện dòng lệnh CLI.....	23
7.2. Giao diện Web GUI.....	23
7.3. Hướng dẫn cài đặt.....	24
7.4. Chạy demo CLI	24
7.5. Chạy demo Web GUI.....	24
7.6. Chạy với mã hóa.....	25
CHƯƠNG 8. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG	26
8.1. Chiến lược kiểm thử	26
8.2. Kiểm thử tự động.....	26

8.3. Kịch bản kiểm thử thủ công CLI.....	27
8.4. Kịch bản kiểm thử Web GUI	27
8.5. Đánh giá mức độ đáp ứng đề bài.....	28
CHƯƠNG 9. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	29
9.1. Kết luận	29
9.2. Ưu điểm của hệ thống	29
9.3. Hạn chế.....	29
9.4. Hướng phát triển.....	30
PHỤ LỤC A. DANH SÁCH LỆNH VÀ CẤU HÌNH DEMO	31
A.1. Lệnh chạy hệ thống cơ bản.....	31
A.2. Lệnh chạy với Web GUI	31
A.3. Lệnh chạy với mã hóa	31
A.4. Lệnh kiểm thử.....	31
A.5. Kịch bản demo bảo vệ	32
TÀI LIỆU THAM KHẢO	33

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa	Giải thích trong đề án
P2P	Peer-to-Peer	Mô hình ngang hàng, mỗi nút vừa gửi vừa nhận dữ liệu.
TCP	Transmission Control Protocol	Giao thức truyền tin đáng tin cậy theo luồng byte, dùng cho kết nối trực tiếp giữa các peer.
ACK	Acknowledgement	Thông điệp xác nhận peer nhận đã nhận được tin nhắn.
GUI	Graphical User Interface	Giao diện đồ họa người dùng trên trình duyệt web.
E2EE	End-to-End Encryption	Mã hóa đầu-cuối nội dung tin nhắn.
NDJSON	Newline-Delimited JSON	Định dạng JSON mỗi thông điệp kết thúc bằng ký tự xuống dòng.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Đề tài “Phát triển hệ thống chat ngang hàng P2P (Peer-to-Peer Chat System)” thuộc môn Các Hệ Thống Phân Tán. Mục tiêu của đề tài là xây dựng một hệ thống chat cho phép nhiều người dùng trao đổi tin nhắn trực tiếp qua mạng, không phụ thuộc hoàn toàn vào máy chủ trung tâm. Trong hệ thống, mỗi peer vừa đóng vai trò client để gửi tin nhắn, vừa đóng vai trò server để tiếp nhận tin nhắn từ các peer khác.

Hệ thống được triển khai theo mô hình Hybrid P2P. Bootstrap/Tracker Server chỉ hỗ trợ đăng ký, khám phá peer và theo dõi trạng thái online/offline. Tin nhắn chat chính được truyền trực tiếp giữa các peer thông qua TCP Socket. Cách thiết kế này giúp hệ thống thể hiện rõ các đặc điểm của hệ thống phân tán: nhiều tiến trình độc lập, giao tiếp qua mạng, quản lý trạng thái phân tán và xử lý lỗi khi peer rời mạng.

1.2. Mục tiêu thực hiện

- Xây dựng được hệ thống chat ngang hàng trong đó mỗi peer có khả năng gửi và nhận tin nhắn trực tiếp qua TCP Socket.
- Thiết kế Bootstrap/Tracker Server đơn giản để hỗ trợ peer discovery và quản lý danh sách peer online.
- Hiện thực các chức năng bắt buộc gồm tham gia mạng, chat trực tiếp, chat nhóm, peer discovery, trạng thái online/offline và truyền tin đáng tin cậy.
- Bổ sung các chức năng nâng cao gồm broadcast toàn mạng, store-and-forward khi peer offline, mã hóa tin nhắn, Web GUI và mô phỏng churn.
- Xây dựng kịch bản kiểm thử để chứng minh hệ thống hoạt động đúng trong các tình huống bình thường và khi xảy ra lỗi.

1.3. Phạm vi hệ thống

Đồ án tập trung vào việc mô phỏng và hiện thực các khái niệm cốt lõi của hệ thống phân tán ở môi trường cục bộ hoặc mạng LAN. Các peer được chạy như các tiến trình độc lập trên các cổng khác nhau. Bootstrap Server không xử lý nội dung chat và không đóng vai trò trung gian truyền tin nhắn. Các chức năng nâng cao được triển khai ở mức phù hợp

cho bài tập lớn, ưu tiên khả năng chạy demo, giải thích kiến trúc và kiểm chứng bằng test case.

1.4. Tóm tắt kết quả đạt được

Nhóm chức năng	Kết quả triển khai
Chức năng cơ bản	Đã triển khai tham gia mạng, lấy danh sách peer online, chat trực tiếp, chat nhóm, peer discovery, trạng thái online/offline và ACK + retry.
Yêu cầu kỹ thuật	Dùng TCP Socket cho giao tiếp peer-to-peer, mỗi peer vừa mở TCP server vừa dùng TCP client, xử lý nhiều kết nối đồng thời.
Chức năng nâng cao	Đã triển khai broadcast, store-and-forward, mã hóa AES-256-CBC, Web GUI và churn simulation.
Kiểm thử	Có bộ kiểm thử tự động gồm 9 test suite, 38 test case và các kịch bản kiểm thử thủ công CLI/Web GUI.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Hệ thống phân tán

Hệ thống phân tán là hệ thống gồm nhiều tiến trình hoặc nút tính toán độc lập, phối hợp với nhau thông qua mạng truyền thông để thực hiện một mục tiêu chung. Đối với đề án này, mỗi peer là một tiến trình độc lập, có cổng TCP riêng, có trạng thái riêng và giao tiếp với các peer khác bằng thông điệp qua mạng.

2.2. Mô hình Peer-to-Peer

Trong mô hình peer-to-peer, các nút mạng có vai trò ngang hàng. Mỗi peer không chỉ sử dụng dịch vụ mà còn cung cấp dịch vụ cho peer khác. Áp dụng vào hệ thống chat, mỗi peer vừa là client khi gửi tin, vừa là server khi lắng nghe kết nối TCP đến. Mô hình này khác với mô hình client-server truyền thống vì tin nhắn chat không cần đi qua server trung tâm.

Đề án sử dụng mô hình Hybrid P2P. Bootstrap Server vẫn tồn tại nhưng chỉ làm nhiệm vụ hỗ trợ khám phá peer và quản lý trạng thái, không relay nội dung chat. Việc này giúp hệ thống giải quyết bài toán cold start: peer mới tham gia chưa biết danh sách peer khác nên cần một tracker ban đầu.

2.3. TCP Socket

TCP Socket được sử dụng để truyền thông trực tiếp giữa các peer. Mỗi peer mở một TCP server bằng module net của Node.js để nhận tin nhắn. Khi cần gửi tin, peer tạo TCP client kết nối tới host và port của peer đích. TCP đảm bảo thứ tự byte trong một kết nối, nhưng không thay thế được cơ chế xác nhận ở tầng ứng dụng. Vì vậy hệ thống bổ sung ACK, timeout và retry để xác định tin nhắn đã được peer đích xử lý hay chưa.

2.4. Peer Discovery và Heartbeat

Peer discovery là cơ chế giúp một peer tìm thấy các peer khác trong mạng. Trong hệ thống này, peer khi khởi động gửi thông tin id, name, host và port đến Bootstrap Server thông qua API /register. Khi muốn gửi tin, peer gọi API /peers để lấy danh sách peer online, sau đó mở kết nối TCP trực tiếp tới peer đích. Peer duy trì trạng thái online

bằng cách gửi heartbeat định kỳ 5 giây/lần. Nếu quá 15 giây không có heartbeat, Bootstrap Server coi peer đó là offline.

2.5. Công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Runtime	Node.js	Chạy Bootstrap Server và các Peer Node.
Giao tiếp P2P	TCP Socket / net module	Gửi nhận tin nhắn trực tiếp giữa các peer.
Tracker	Express.js	Xây dựng REST API phục vụ register, peers, heartbeat, leave, store.
Web GUI	Express.js + Socket.IO	Cung cấp giao diện web và cập nhật thời gian thực trong trình duyệt.
Mã hóa	Node.js crypto	Mã hóa và giải mã AES-256-CBC.
Kiểm thử	Node.js test script	Chạy bộ test tự động cho API, TCP, ACK, encryption, store-and-forward và churn.

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU HỆ THỐNG

3.1. Yêu cầu chức năng theo đề bài

Mã yêu cầu	Nội dung yêu cầu	Cách hệ thống đáp ứng
FR1	Tham gia mạng P2P	Peer đăng ký với Bootstrap Server thông qua POST /register và được đưa vào danh sách peer online.
FR2	Chat trực tiếp giữa các peer	Peer gửi tin trực tiếp tới peer đích bằng TCP Socket dựa trên host:port lấy từ /peers.
FR3	Chat nhóm	Lệnh /group gửi cùng một nội dung tới nhiều peer được chỉ định; mỗi peer nhận và ACK riêng.
FR4	Peer discovery	Peer mới gọi /peers để tìm các peer khác trong mạng.
FR5	Trạng thái online/offline	Heartbeat 5 giây/lần, timeout 15 giây, /leave khi rời mạng chủ động.
FR6	Truyền tin đáng tin cậy	ACK + timeout 5 giây + retry tối đa 3 lần; hết retry thì failed/store-and-forward.

3.2. Yêu cầu kỹ thuật

Yêu cầu kỹ thuật	Hiện thực trong hệ thống
Giao tiếp mạng bằng TCP socket hoặc tương đương	Dùng TCP Socket thông qua module net của Node.js cho đường truyền chính giữa các peer.
Mỗi peer gửi và nhận tin đồng thời	Mỗi peer có TCP server để nhận tin và TCP client để gửi tin; các hoạt động được xử lý bất đồng bộ.
Xử lý nhiều kết nối cùng lúc	TCP server của peer có thể tiếp nhận nhiều socket connection từ các peer khác.

Yêu cầu kỹ thuật	Hiện thực trong hệ thống
Quản lý danh sách peer đang kết nối	Bootstrap Server duy trì peer registry dạng Map và cập nhật bằng register/heartbeat/leave/timeout.
Có Peer Node	peer.js và các module liên quan đóng vai trò peer độc lập.
Có Bootstrap/Tracker Server	Express server cung cấp REST API cho peer discovery và trạng thái.

3.3. Chức năng nâng cao

Chức năng nâng cao	Trạng thái	Mô tả
Broadcast message trong toàn mạng	Đã triển khai	Lệnh /broadcast gửi tin tới tất cả peer online trừ peer gửi.
Store-and-forward khi peer offline	Đã triển khai	Khi peer đích offline và retry thất bại, tin được lưu tạm và chuyển khi peer đăng ký lại.
Mã hóa tin nhắn	Đã triển khai	AES-256-CBC với khóa chia sẻ trước, nội dung content được mã hóa dạng enc:<iv>:<ciphertext>.
Giao diện GUI/Web interface	Đã triển khai	Web GUI chạy tại port TCP + 1000, dùng Socket.IO để cập nhật giao diện.
Mô phỏng churn	Đã triển khai	Script churn-sim.js tự động tạo peer, tắt peer và kiểm tra khả năng phục hồi.
File transfer giữa các peer	Chưa triển khai trong phiên bản hiện tại	Được đưa vào hướng phát triển tiếp theo bằng TCP stream hoặc chunked transfer.

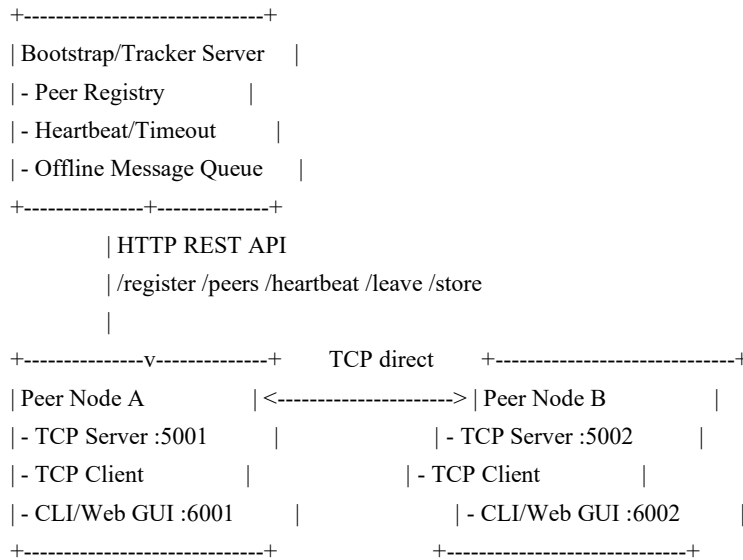
3.4. Yêu cầu phi chức năng

- Dễ chạy demo: hệ thống chạy local bằng nhiều terminal, không yêu cầu hạ tầng phức tạp.
- Dễ kiểm chứng: mỗi chức năng có lệnh CLI hoặc thao tác GUI tương ứng để trình diễn.
- Tính mô-đun: các phần TCP server, TCP client, message handler, reliable delivery, bootstrap client và GUI được tách thành module riêng.
- Khả năng chịu lỗi cơ bản: có heartbeat timeout, retry, failed status, deduplication và store-and-forward.
- Tính minh bạch kiến trúc: Bootstrap không truyền tin nhắn chat, chỉ hỗ trợ discovery và trạng thái.

CHƯƠNG 4. KIẾN TRÚC VÀ THIẾT KẾ HỆ THỐNG

4.1. Kiến trúc tổng thể

Hệ thống gồm hai nhóm thành phần chính: Bootstrap/Tracker Server và các Peer Node. Bootstrap Server là thành phần hỗ trợ khám phá mạng, lưu danh sách peer online và hàng đợi tin nhắn offline. Peer Node là thành phần trung tâm của hệ thống chat, vừa nhận tin bằng TCP server, vừa gửi tin bằng TCP client.



4.2. Bootstrap/Tracker Server

Bootstrap Server được xây dựng bằng Express.js. Thành phần này không tham gia vào nội dung hội thoại giữa các peer. Vai trò của Bootstrap Server gồm:

- Nhận đăng ký peer khi peer tham gia mạng.
- Cung cấp danh sách peer online cho các peer khác.
- Cập nhật lastSeen thông qua heartbeat định kỳ.
- Xóa peer khỏi danh sách khi peer rời mạng hoặc quá thời gian timeout.
- Lưu hàng đợi tin nhắn offline trong chức năng store-and-forward.

4.3. Peer Node

Mỗi Peer Node là một tiến trình Node.js độc lập, được khởi chạy với id, name và port riêng. Peer Node có các nhiệm vụ:

- Mở TCP server trên port được chỉ định để lắng nghe tin nhắn đến.
- Dùng TCP client để kết nối trực tiếp tới peer đích khi gửi tin.
- Gọi Bootstrap Server để đăng ký, heartbeat, lấy danh sách peer và rời mạng.
- Xử lý các loại thông điệp CHAT, GROUP_CHAT, BROADCAST, ACK và ERROR.
- Quản lý pending ACK, retry, deduplication, encryption và offline messages.

4.4. Cấu trúc thư mục và module

```
Chat P2P/
|-- bootstrap-server/
|   |-- server.js          # Express HTTP REST API
|   |-- peerRegistry.js    # Quản lý peer registry và offline queue
|   |
|-- peer-node/
|   |-- peer.js            # Entry point, khởi tạo peer
|   |-- tcpServer.js       # TCP server nhận tin
|   |-- tcpClient.js       # TCP client gửi tin
|   |-- messageHandler.js  # Xử lý CHAT/GROUP/BROADCAST/ACK/ERROR
|   |-- reliableDelivery.js # ACK + retry + store-and-forward trigger
|   |-- bootstrapClient.js # HTTP client gọi Bootstrap Server
|   |-- crypto.js          # AES-256-CBC encrypt/decrypt
|   |-- state.js           # Shared state: pendingAcks, receivedMsgIds
|   |-- eventBus.js        # Event bus cho Web GUI
|   |-- cli.js             # Giao diện dòng lệnh
|   |-- webServer.js       # Web GUI server
|   |-- public/            # HTML/CSS/JS frontend
|   |
|-- churn-sim.js           # Mô phỏng peer churn
|-- test.js                # Bộ kiểm thử tự động
|-- README.md              # Hướng dẫn chạy
|-- REPORT.md              # Báo cáo kỹ thuật
```

4.5. Luồng hoạt động tổng quát

1. Peer khởi động và mở TCP server tại port được cấu hình.
2. Peer gửi POST /register tới Bootstrap Server để đăng ký thông tin id, name, host và port.
3. Peer gửi heartbeat định kỳ tới Bootstrap Server để duy trì trạng thái online.
4. Khi người dùng gửi tin, peer gọi GET /peers để tìm host và port của peer đích.
5. Peer mở TCP connection trực tiếp tới peer đích, gửi JSON message và chờ ACK.

6. Peer nhận xử lý message, hiển thị nội dung và gửi ACK về trên cùng socket.
7. Nếu không nhận được ACK hoặc không kết nối được, peer gửi retry; hết retry thì báo failed hoặc lưu store-and-forward.

CHƯƠNG 5. GIAO THỨC TRAO ĐỔI THÔNG ĐIỆN

5.1. Giao thức REST giữa Peer và Bootstrap Server

Method	Endpoint	Chức năng	Request/Response chính
POST	/register	Đăng ký peer vào mạng	Request: {id, name, host, port}. Response: {ok, pendingMessages}.
GET	/peers	Lấy danh sách peer online	Response: {peers: [{id, name, host, port}]}.
POST	/heartbeat	Cập nhật trạng thái sống	Request: {id}. Response: {ok} hoặc 404 nếu peer chưa đăng ký.
POST	/leave	Rời mạng chủ động	Request: {id}. Bootstrap xóa peer khỏi registry.
POST	/store	Lưu tin nhắn cho peer offline	Request: {to, from, payload}. Response: {ok, queueSize} hoặc 409 nếu peer online.
GET	/	Health check	Trả về trạng thái server, số peer online và thông tin tổng quan.

5.2. Giao thức TCP giữa các peer

Thông điệp TCP giữa các peer được biểu diễn dưới dạng JSON và kết thúc bằng ký tự xuống dòng “\n”. Cách đóng gói này gọi là Newline-Delimited JSON. Vì TCP là stream protocol, dữ liệu có thể bị chia thành nhiều chunk; ký tự xuống dòng giúp peer nhận xác định ranh giới thông điệp để parse đúng JSON hoàn chỉnh.

<JSON string>

Ví dụ:


```
{"type": "CHAT", "id": "msg-001", "from": "peer-a", "to": "peer-b", "content": "Hello", "timestamp": 1700000000000}
```

Luồng kết nối:

Peer A mở TCP connection -> gửi JSON -> giữ socket để chờ ACK -> Peer B gửi ACK -> đóng socket

5.3. Các loại thông điệp

Loại thông điệp	Mục đích	Trường dữ liệu chính
CHAT	Tin nhắn trực tiếp 1-1	type, id, from, to, content, timestamp
GROUP_CHAT	Tin nhắn nhóm tới danh sách peer	type, id, from, to[], content, timestamp
BROADCAST	Tin nhắn phát tới tất cả peer online	type, id, from, content, timestamp
ACK	Xác nhận peer nhận đã nhận tin	type, id, from
ERROR	Phản hồi lỗi khi thông điệp không hợp lệ	type, id, reason

5.4. Mẫu JSON thông điệp

```
// Tin nhắn trực tiếp
{
  "type": "CHAT",
  "id": "msg-1700000000000-abcd",
  "from": "peer-a",
  "to": "peer-b",
  "content": "Hello Hieu!",
  "timestamp": 1700000000000
}
```

```
// Tin nhắn nhóm
{
  "type": "GROUP_CHAT",
  "id": "msg-1700000000001-efgh",
  "from": "peer-a",
  "to": ["peer-b", "peer-c"],
}
```

```
"content": "Xin chào cả nhóm!",  
"timestamp": 1700000000001  
}
```

// Broadcast

```
{  
  "type": "BROADCAST",  
  "id": "msg-1700000000002-ijkl",  
  "from": "peer-a",  
  "content": "Xin chào tất cả!",  
  "timestamp": 1700000000002  
}
```

// ACK

```
{  
  "type": "ACK",  
  "id": "msg-1700000000000-abcd",  
  "from": "peer-b"  
}
```

5.5. Luồng gửi tin nhắn trực tiếp

Peer A	Bootstrap	Peer B
--- GET /peers ----->		
<--- danh sách peer B -----		
--- TCP connect 127.0.0.1:5002 ----->		
--- CHAT JSON ----->		
<--- ACK JSON -----		
--- close socket ----->		

CHƯƠNG 6. CÁC CƠ CHẾ XỬ LÝ PHÂN TÁN VÀ CHỊU LỖI

6.1. Cơ chế peer discovery

Cơ chế peer discovery được hiện thực theo mô hình tracker. Bootstrap Server lưu danh sách peer đang online trong cấu trúc Map. Khi peer muốn chat, peer không gửi tin qua Bootstrap mà chỉ dùng Bootstrap để lấy địa chỉ của peer đích.

8. Peer gửi POST /register khi tham gia mạng.
9. Bootstrap lưu thông tin peer và thời điểm lastSeen.
10. Peer gọi GET /peers để lấy danh sách peer online.
11. Peer mở TCP connection trực tiếp tới host:port của peer đích.
12. Peer gửi POST /heartbeat định kỳ để duy trì trạng thái online.
13. Peer gọi POST /leave khi rời mạng chủ động hoặc bị timeout khi mất heartbeat.

6.2. Quản lý trạng thái online/offline

Mỗi peer gửi heartbeat 5 giây/lần. Bootstrap Server cập nhật trường lastSeen cho peer tương ứng. Nếu peer không gửi heartbeat trong 15 giây, Bootstrap coi peer đó đã offline và loại khỏi danh sách online. Khi peer rời mạng bằng lệnh /exit, peer chủ động gọi /leave để cập nhật trạng thái ngay.

Cơ chế	Giá trị	Ý nghĩa
Heartbeat interval	5 giây	Tần suất peer báo hiệu còn sống.
Peer timeout	15 giây	Sau 3 lần mất heartbeat, peer bị coi là offline.
Periodic cleanup	10 giây	Bootstrap định kỳ dọn peer timeout và tin nhắn hết hạn.
Graceful leave	/leave	Peer rời mạng chủ động không cần chờ timeout.

6.3. Cơ chế truyền tin đáng tin cậy

Dù TCP đảm bảo byte được truyền theo thứ tự trong một kết nối, hệ thống vẫn cần biết peer đích đã xử lý tin nhắn hay chưa. Vì vậy hệ thống xây dựng cơ chế ACK ở tầng ứng dụng. Khi gửi một message, peer gửi lưu message id vào pendingAcks, đặt timer 5 giây và chờ ACK. Nếu hết thời gian mà chưa có ACK, peer tự động gửi lại tối đa 3 lần. Nếu sau các lần retry vẫn thất bại, message được đánh dấu failed và kích hoạt store-and-forward nếu peer đích offline.

```
sendWithAck(payload)
-> lưu pendingAcks[msgId]
-> gửi TCP tới peer đích
-> chờ ACK trong 5 giây
-> nhận ACK: clear timer, xóa pendingAcks, trạng thái DELIVERED
-> timeout: retry nếu retryCount < 3
-> hết retry: FAILED, kích hoạt Store-and-Forward nếu phù hợp
```

6.4. Khử trùng lặp tin nhắn

Do cơ chế retry, peer nhận có thể nhận cùng một message id nhiều lần. Để tránh hiển thị trùng tin nhắn, peer sử dụng receivedMsgIds dạng Set. Nếu message id đã tồn tại trong Set, peer bỏ qua việc hiển thị nhưng vẫn gửi ACK để peer gửi dừng retry. Kích thước Set được giới hạn để tránh tăng bộ nhớ không kiểm soát.

6.5. Store-and-Forward khi peer offline

Store-and-forward giúp cải thiện khả năng chịu lỗi khi peer đích offline. Sau khi gửi TCP thất bại và retry hết số lần, peer gửi gọi POST /store tới Bootstrap Server. Bootstrap chỉ lưu tin nếu peer đích đang offline. Khi peer đích online trở lại và gọi /register, Bootstrap trả về danh sách pendingMessages để peer nhận hiển thị tin offline.

```
Peer A gửi cho Peer C offline
-> TCP ECONNREFUSED
-> retry 1/3, 2/3, 3/3
-> FAILED
-> POST /store {to: "peer-c", from: "peer-a", payload}
-> Bootstrap lưu vào offlineQueue[peer-c]
```

```
Peer C online lại
-> POST /register {id: "peer-c", host, port}
```

- > Bootstrap popMessages("peer-c")
- > Peer C nhận và hiển thị offline messages

Giới hạn	Giá trị	Mục đích
Số tin tối đa/peer	50 tin	Tránh Bootstrap bị quá tải RAM.
TTL tin nhắn	1 giờ	Loại bỏ tin quá cũ, không còn giá trị demo.
Kiểm tra online	409 Conflict	Nếu peer đang online thì không store, yêu cầu gửi TCP trực tiếp.

6.6. Mã hóa tin nhắn AES-256-CBC

Hệ thống hỗ trợ mã hóa nội dung tin nhắn bằng AES-256-CBC với khóa chia sẻ trước. Khi chạy peer với tham số --key, nội dung content được mã hóa trước khi gửi qua TCP. Peer nhận có cùng khóa sẽ giải mã được nội dung; nếu sai khóa hoặc không có khóa thì hệ thống hiển thị thông báo lỗi giải mã.

Định dạng content đã mã hóa:
enc:<iv_hex>:<ciphertext_hex>

Luồng xử lý:
plaintext -> deriveKey(passphrase bằng SHA-256) -> random IV 16 bytes -> AES-256-CBC encrypt -> gửi qua TCP
ciphertext -> tách IV/ciphertext -> deriveKey(passphrase) -> AES-256-CBC decrypt -> plaintext

6.7. Broadcast và chat nhóm

Chat nhóm được triển khai bằng cách peer gửi lặp message tới từng peer trong danh sách nhóm. Broadcast là trường hợp mở rộng, peer gửi lấy danh sách toàn bộ peer online từ Bootstrap và gửi trực tiếp tới từng peer. Với cách làm này, Bootstrap vẫn không relay tin nhắn; peer gửi chịu trách nhiệm mở các kết nối TCP trực tiếp tới các peer nhận.

6.8. Mô phỏng churn

Churn mô tả hiện tượng peer liên tục tham gia và rời mạng trong hệ thống phân tán. Script churn-sim.js tự động tạo nhiều peer, cho peer hoạt động trong một khoảng thời

gian, sau đó tắt các tiến trình peer và kiểm tra Bootstrap Server có dọn trạng thái đúng hay không. Chức năng này giúp kiểm thử khả năng tự phục hồi của cơ chế heartbeat và timeout.

CHƯƠNG 7. GIAO DIỆN NGƯỜI DÙNG VÀ HƯỚNG DẪN CHẠY

7.1. Giao diện dòng lệnh CLI

CLI là giao diện chính để chạy demo kỹ thuật. Mỗi peer có prompt riêng, cho phép người dùng xem danh sách peer, gửi tin trực tiếp, gửi nhóm, broadcast và rời mạng. CLI cũng hiển thị trạng thái ACK, retry, failed, encrypted message và offline message.

Lệnh	Ý nghĩa	Ví dụ
/help	Hiển thị danh sách lệnh	/help
/list	Lấy danh sách peer đang online	/list
/msg <peer-id> <nội dung>	Gửi tin 1-1	/msg peer-b Hello!
/group <a,b> <nội dung>	Gửi tin nhóm	/group peer-b,peer- c Xin chào nhóm
/broadcast <nội dung>	Gửi tin tới tất cả peer online	/broadcast Hello all
/status	Xem trạng thái peer hiện tại	/status
/exit	Rời mạng và tắt peer	/exit

7.2. Giao diện Web GUI

Web GUI được chạy khi peer khởi động với tham số --gui true. Mỗi GUI chạy ở cổng bằng TCP port + 1000. Ví dụ peer-a dùng TCP port 5001 thì Web GUI chạy tại http://localhost:6001. GUI hỗ trợ danh sách peer thời gian thực, khung chat, trạng thái gửi/ACK, thông báo tin mới, badge mã hóa và lịch sử cục bộ trên trình duyệt.

- Danh sách peer online cập nhật định kỳ.
- Hiển thị tin nhắn theo từng cuộc trò chuyện.

- Hiện thị trạng thái gửi: đang gửi, đã nhận ACK, thất bại.
- Có thông báo toast cho tin mới, lỗi gửi và tin offline.
- Hỗ trợ chạy với encryption key và hiện thị trạng thái AES-256.

7.3. Hướng dẫn cài đặt

```
# Cài đặt Bootstrap Server
cd bootstrap-server
npm install
```

```
# Cài đặt Peer Node
cd ../peer-node
npm install
```

7.4. Chạy demo CLI

```
# Terminal 1 - Bootstrap Server
cd bootstrap-server
npm run dev
```

```
# Terminal 2 - Peer A
cd peer-node
node peer.js --id peer-a --name Dung --port 5001
```

```
# Terminal 3 - Peer B
cd peer-node
node peer.js --id peer-b --name Hieu --port 5002
```

```
# Terminal 4 - Peer C
cd peer-node
node peer.js --id peer-c --name Viet --port 5003
```

7.5. Chạy demo Web GUI

```
cd peer-node
node peer.js --id peer-a --name Dung --port 5001 --gui true
node peer.js --id peer-b --name Hieu --port 5002 --gui true
node peer.js --id peer-c --name Viet --port 5003 --gui true
```

Truy cập trình duyệt:

Peer A: <http://localhost:6001>

Peer B: <http://localhost:6002>

Peer C: <http://localhost:6003>

7.6. Chạy với mã hóa

```
node peer.js --id peer-a --name Dung --port 5001 --key dung123
```

```
node peer.js --id peer-b --name Hieu --port 5002 --key dung123
```

```
node peer.js --id peer-c --name Viet --port 5003 --key dung123
```

```
# Kết hợp Web GUI và encryption
```

```
node peer.js --id peer-a --name Dung --port 5001 --gui true --key dung123
```

CHƯƠNG 8. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

8.1. Chiến lược kiểm thử

Hệ thống được kiểm thử theo hai hướng: kiểm thử tự động bằng script test.js và kiểm thử thủ công thông qua CLI/Web GUI. Kiểm thử tự động giúp xác nhận các module chính hoạt động đúng. Kiểm thử thủ công giúp chứng minh hệ thống chạy được trong demo thực tế với nhiều terminal và trình duyệt.

8.2. Kiểm thử tự động

Test suite	Số test	Mục tiêu kiểm thử
Bootstrap REST API	8	Đăng ký peer, lấy danh sách peer, heartbeat, leave, validate request, health check.
TCP CHAT -> ACK	2	Peer nhận CHAT và peer gửi nhận ACK.
GROUP_CHAT -> ACK	4	Nhiều peer nhận group chat và gửi ACK tương ứng.
BROADCAST -> ACK	2	Peer nhận broadcast và phản hồi ACK.
ACK timeout	1	Kết nối tới peer không reachable, kích hoạt retry/failed.
Message Deduplication	2	Xử lý message id trùng lặp.
Peer Churn Simulation	5	Peer join, leave, rejoin và nhiều peer online.
Store-and-Forward	6	Queue tin offline, trả tin khi reconnect, không duplicate.
AES-256-CBC Encryption	8	Encrypt/decrypt, wrong key, IV ngẫu nhiên, Unicode.

Kết quả kiểm thử tự động: 38 test cases passed, 0 failed.

```
# 1. Khởi động Bootstrap Server trên cổng riêng
$env:PORT=3001; node bootstrap-server/server.js
```

```
# 2. Chạy bộ kiểm thử
$env:BOOTSTRAP_PORT=3001; node test.js
```

Kết quả mong đợi:
Result: 38 passed, 0 failed

8.3. Kịch bản kiểm thử thủ công CLI

14. Chạy Bootstrap Server.
15. Chạy Peer A, Peer B, Peer C ở ba terminal khác nhau.
16. Tại Peer A chạy /list để xác nhận Peer B và Peer C online.
17. Tại Peer A chạy /msg peer-b Hello B để kiểm tra chat trực tiếp và ACK.
18. Tại Peer A chạy /group peer-b,peer-c Hello group để kiểm tra chat nhóm.
19. Tại Peer A chạy /broadcast Hello all để kiểm tra broadcast toàn mạng.
20. Tắt Peer C bằng Ctrl+C và chờ timeout.
21. Tại Peer A gửi /msg peer-c Are you there? để kiểm tra retry, failed và store-and-forward.
22. Khởi động lại Peer C để kiểm tra nhận offline message.

8.4. Kịch bản kiểm thử Web GUI

23. Chạy các peer với tham số --gui true.
24. Mở trình duyệt tại các địa chỉ <http://localhost:6001>, <http://localhost:6002>, <http://localhost:6003>.
25. Kiểm tra danh sách peer online trên sidebar.
26. Gửi tin nhắn 1-1 từ Peer A tới Peer B và quan sát trạng thái ACK.
27. Gửi broadcast và kiểm tra thông báo ở các peer còn lại.
28. Tắt một peer, gửi tin cho peer đó và kiểm tra thông báo failed/store.
29. Khởi động lại peer offline để kiểm tra tin nhắn được chuyển tiếp.

8.5. Đánh giá mức độ đáp ứng đề bài

Tiêu chí	Đánh giá
Mô hình peer-to-peer	Đạt. Tin nhắn chính đi trực tiếp peer-to-peer qua TCP, không qua Bootstrap.
Truyền thông giữa tiến trình	Đạt. Mỗi peer là một process độc lập giao tiếp qua TCP/HTTP.
Peer discovery	Đạt. Bootstrap Server cung cấp registry và API /peers.
Đồng bộ và quản lý trạng thái phân tán	Đạt. Peer registry, heartbeat, timeout, leave và trạng thái pending ACK.
Xử lý lỗi và chịu lỗi cơ bản	Đạt. Retry, failed status, deduplication, store-and-forward và churn simulation.
Chức năng nâng cao	Đạt phần lớn. Đã có broadcast, store-and-forward, encryption, GUI và churn; file transfer là hướng phát triển.

CHƯƠNG 9. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

9.1. Kết luận

Đồ án đã xây dựng được một hệ thống chat ngang hàng P2P đáp ứng các yêu cầu chính của chủ đề. Hệ thống sử dụng Bootstrap Server để hỗ trợ khám phá peer nhưng không phụ thuộc hoàn toàn vào server trung tâm trong quá trình trao đổi tin nhắn. Các peer gửi và nhận tin trực tiếp qua TCP Socket, có cơ chế xác nhận ACK, timeout, retry và xử lý khi peer offline.

Bên cạnh các chức năng bắt buộc, hệ thống cũng triển khai nhiều chức năng nâng cao như broadcast toàn mạng, store-and-forward, mã hóa AES-256-CBC, Web GUI và mô phỏng churn. Các chức năng này giúp đồ án thể hiện rõ hơn các vấn đề thực tế của hệ thống phân tán như peer join/leave, mất kết nối, tin nhắn trùng lặp, độ tin cậy và bảo mật nội dung truyền qua mạng.

9.2. Ưu điểm của hệ thống

- Kiến trúc đúng định hướng P2P: Bootstrap chỉ phục vụ discovery, không relay tin nhắn chat.
- Mỗi peer vừa là client vừa là server, thể hiện rõ mô hình ngang hàng.
- Giao thức JSON qua TCP đơn giản, dễ debug, dễ trình bày trong báo cáo.
- Có cơ chế truyền tin đáng tin cậy ở tầng ứng dụng với ACK, timeout và retry.
- Có các chức năng nâng cao giúp tăng điểm demo: broadcast, encryption, GUI, store-and-forward và churn simulation.
- Có bộ kiểm thử tự động và kịch bản demo rõ ràng.

9.3. Hạn chế

- Bootstrap Server vẫn là điểm phụ thuộc cho peer discovery, chưa phải P2P thuần hoàn toàn.
- Store-and-forward hiện lưu trong bộ nhớ, chưa có cơ chế persistence bền vững như SQLite hoặc LevelDB.

- Mã hóa đang dùng khóa chia sẻ trước, chưa có cơ chế trao đổi khóa động như Diffie-Hellman/ECDH.
- Hệ thống chủ yếu chạy local hoặc LAN; chưa xử lý NAT traversal cho môi trường Internet thực tế.
- File transfer chưa được triển khai trong phiên bản hiện tại.

9.4. Hướng phát triển

Hướng phát triển	Mô tả đề xuất
DHT Peer Discovery	Thay Bootstrap bằng Distributed Hash Table như Kademlia để giảm phụ thuộc tracker trung tâm.
Trao đổi khóa động	Dùng ECDH thay cho khóa chia sẻ trước để tăng tính bảo mật.
NAT Traversal	Tích hợp STUN/TURN hoặc hole punching để peer ở mạng khác nhau có thể kết nối.
Persistent Storage	Lưu lịch sử và offline queue bằng SQLite/LevelDB thay vì RAM/localStorage.
Group Encryption	Thiết kế cơ chế quản lý khóa nhóm để mã hóa chat nhóm an toàn hơn.
File Transfer	Bổ sung truyền file qua TCP stream, chia file thành chunk và xác nhận từng phần.
Mobile/Web App hoàn chỉnh	Phát triển client thân thiện hơn bằng React Native, Flutter hoặc web framework hiện đại.

PHỤ LỤC A. DANH SÁCH LỆNH VÀ CẤU HÌNH DEMO

A.1. Lệnh chạy hệ thống cơ bản

```
# Bootstrap Server
cd bootstrap-server
npm install
npm run dev

# Peer A
cd peer-node
npm install
node peer.js --id peer-a --name Dung --port 5001

# Peer B
node peer.js --id peer-b --name Hieu --port 5002

# Peer C
node peer.js --id peer-c --name Viet --port 5003
```

A.2. Lệnh chạy với Web GUI

```
node peer.js --id peer-a --name Dung --port 5001 --gui true
node peer.js --id peer-b --name Hieu --port 5002 --gui true
node peer.js --id peer-c --name Viet --port 5003 --gui true
```

A.3. Lệnh chạy với mã hóa

```
node peer.js --id peer-a --name Dung --port 5001 --key dung123
node peer.js --id peer-b --name Hieu --port 5002 --key dung123
node peer.js --id peer-c --name Viet --port 5003 --key dung123
```

A.4. Lệnh kiểm thử

```
# Bootstrap Server test port
$env:PORT=3001; node bootstrap-server/server.js

# Automated tests
$env:BOOTSTRAP_PORT=3001; node test.js

# Churn simulation
node churn-sim.js --rounds 3
```

A.5. Kịch bản demo bảo vệ

ước	Thao tác	Ý nghĩa chứng minh
	Chạy Bootstrap Server	Có tracker hỗ trợ peer discovery.
	Chạy Peer A, B, C	Có nhiều process độc lập.
	Peer A chạy /list	Danh sách peer online hoạt động.
	Peer A chạy /msg peer-b Hello B	Chat trực tiếp TCP và ACK.
	Peer A chạy /group peer-b,peer-c Hello group	Chat nhóm.
	Peer A chạy /broadcast Hello all	Broadcast toàn mạng.
	Tắt Peer C và gửi tin tới C	Retry, failed và store-and-forward.
	Khởi động lại Peer C	Peer reconnect và nhận offline message.
	Chạy lại với --key	Mã hóa AES-256-CBC.
0	Chạy churn-sim.js	Mô phỏng peer join/leave liên tục.

TÀI LIỆU THAM KHẢO

1. Node.js Documentation - net module, crypto module.
2. Express.js Documentation - REST API server.
3. Socket.IO Documentation - realtime Web GUI communication.